

Đại Học Công Nghiệp Thành Phố Hồ Chí Minh

Môn học : Thực Hành TK Vi Mạch Số với HDL

BÀI 7: MÁY TRẠNG THÁI (FSM)

BÀI 4: MẠCH ĐẾM

1

Giới thiệu chung về FSM

2

Thiết kế FSM theo lưu đồ MOORE

3

Thiết kế FSM có chức năng phát hiện chuỗi

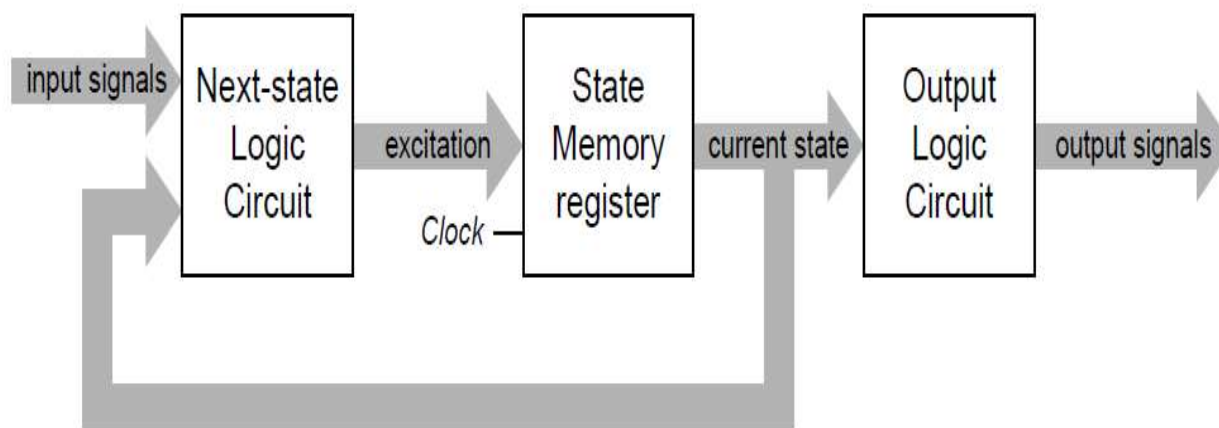
4

Lập trình ứng dụng dùng FSM

1. Giới thiệu chung về FSM

- FSM là một công cụ dùng để mô tả các hành vi của hệ thống số. Nó chỉ ra tất cả các trạng thái có thể có của hệ thống, và những sự kiện nào sẽ gây ra sự thay đổi trạng thái.
- Thành phần cơ bản của FSM là: xung nhịp ck, khối thanh ghi, khối tác động trạng thái kế tiếp, khối tác động ngõ ra
- Số ngõ vào/ra của FSM tùy thuộc vào yêu cầu mạch thiết kế
- Hoạt động của FSM thường được mô tả bằng lưu đồ trạng thái Moore hoặc Mealy

Sơ đồ khối máy trạng thái (FSM)



Next state logic circuit: khối tác động trạng thái kế tiếp

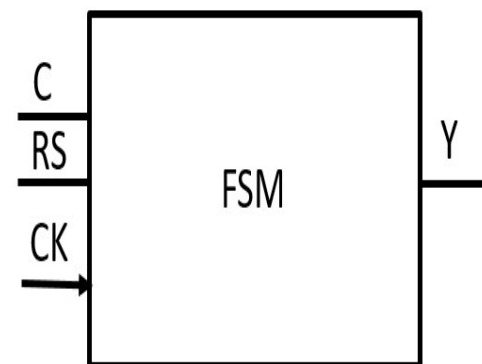
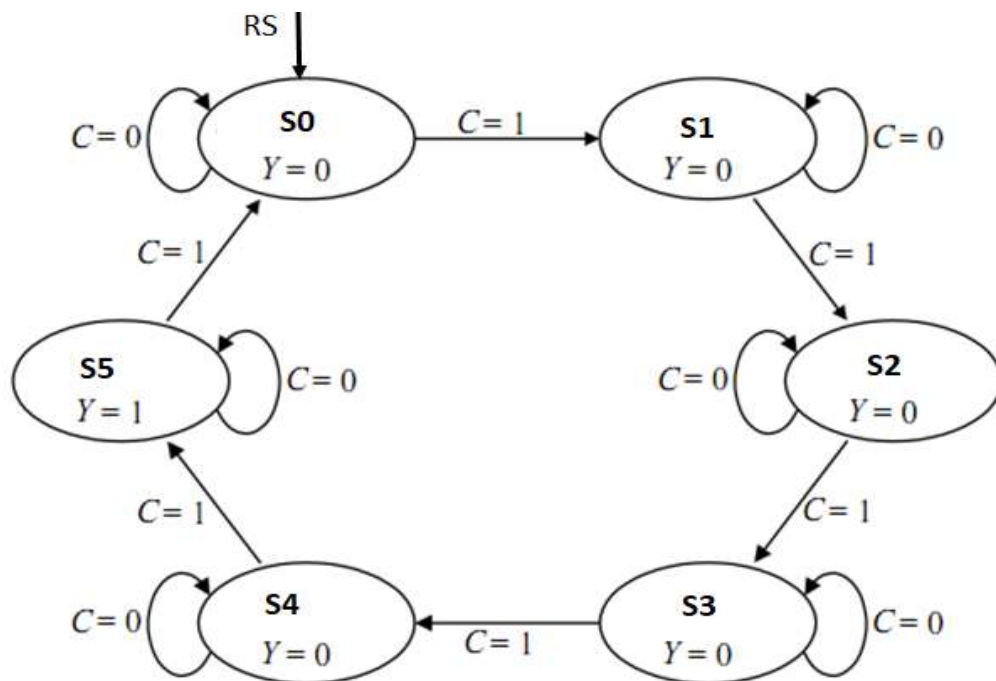
State memory register: khối thanh ghi (lưu trạng thái trong của FSM)

Output logic circuit: khối tác động ngõ ra

2. Các bước thiết kế FSM

- Dựa vào yêu cầu thiết kế hoặc lưu đồ trạng thái xác định số ngõ vào, số ngõ ra, và số trạng thái trong của FSM
- Chọn loại và số lượng flip flop để lưu trạng thái trong của FSM
- Lập bảng chuyển đổi trạng thái
- Rút gọn và viết biểu thức của các ngõ vào của FF và ngõ ra của FSM *theo trạng thái hiện tại*.
- Vẽ mạch thiết kế (nếu cần)
- Viết code theo mô hình hành vi

Ví dụ 1: Mô tả FSM theo giản đồ trạng thái dạng Moore

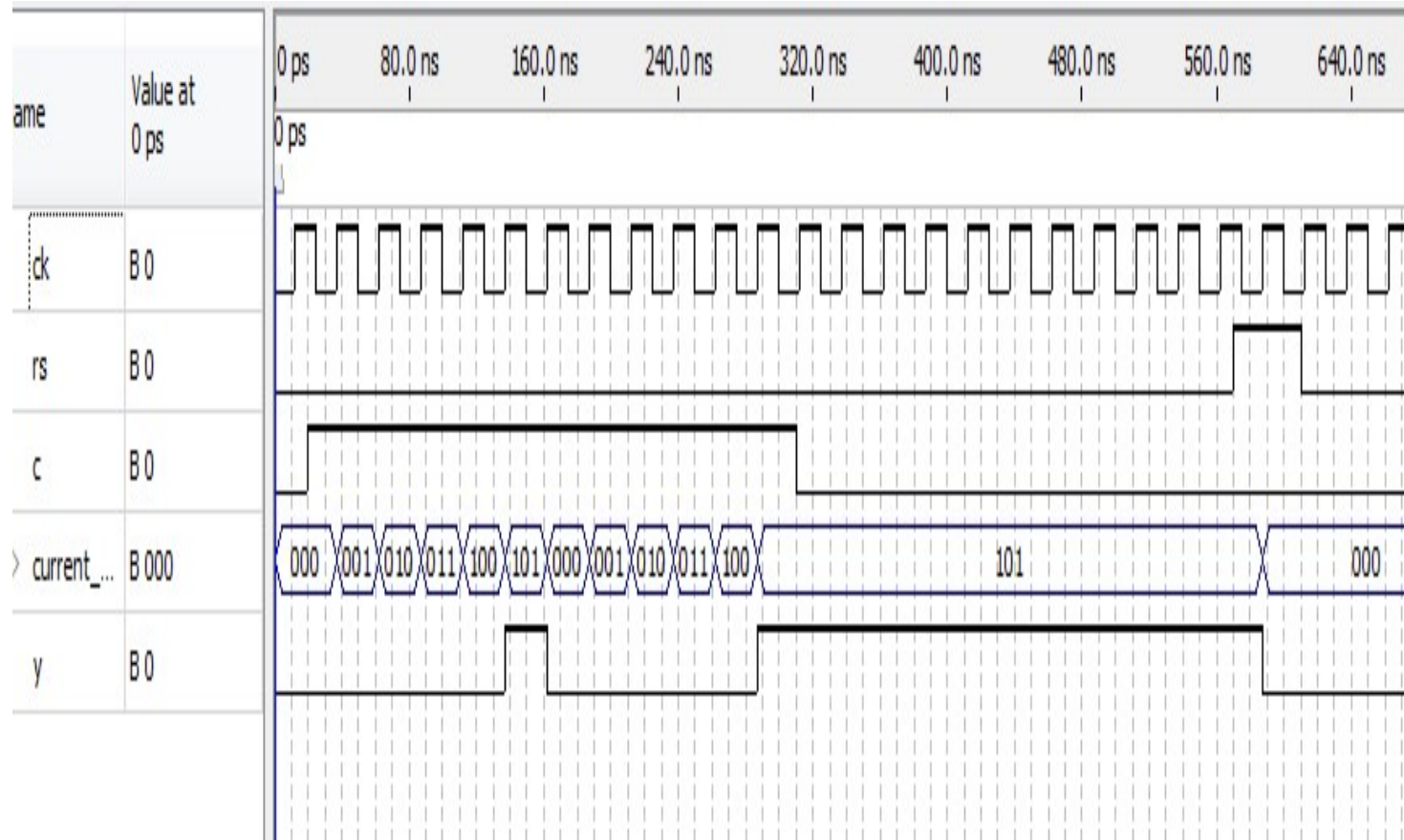


Sơ đồ chân của FSM

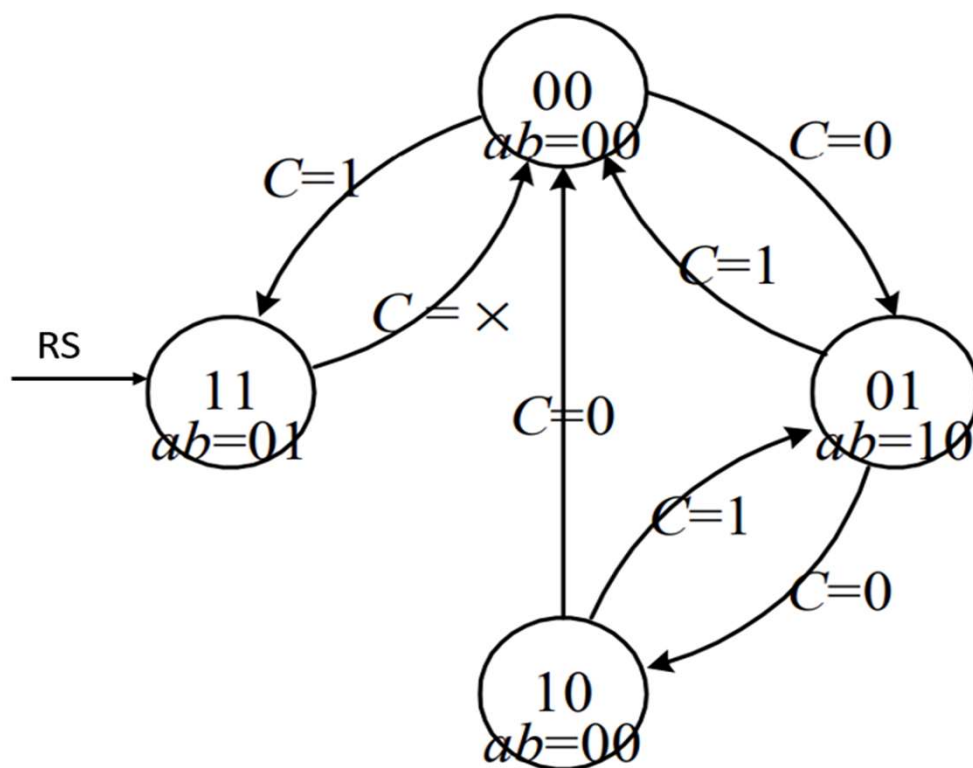
```
module fsm_VD1 (ck,rs,c,y);
parameter s0 = 3'b000;
parameter s1 = 3'b001;
parameter s2 = 3'b010;
parameter s3 = 3'b011;
parameter s4 = 3'b100;
parameter s5 = 3'b101;
input ck,rs,c;
output y;
reg [2:0]current_state, next_state;
reg y;
always @ (*)
begin
case (current_state)
s0 : if (c) next_state = s1;
     else next_state = current_state;
s1 : if (c) next_state = s2;
     else next_state = current_state;
s2 : if (c) next_state = s3;
     else next_state = current_state;
```

```
s3 :   if (c) next_state = s4;
       else next_state = current_state;
s4 :   if (c) next_state = s5;
       else next_state = current_state;
s5 :   if (c) next_state = s0;
       else next_state = current_state;
default next_state = s0;
endcase
end
always @ (posedge ck)
begin
    if (rs == 1) current_state <= s0;
    else current_state <= next_state;
End
always @ (*)
begin
    if (current_state == s5) y = 1'b1;
    else y = 1'b0;
end
Endmodule
```

Kết quả mô phỏng



Ví dụ 2: Viết code verilog mô tả FSM theo giản đồ moore sau:



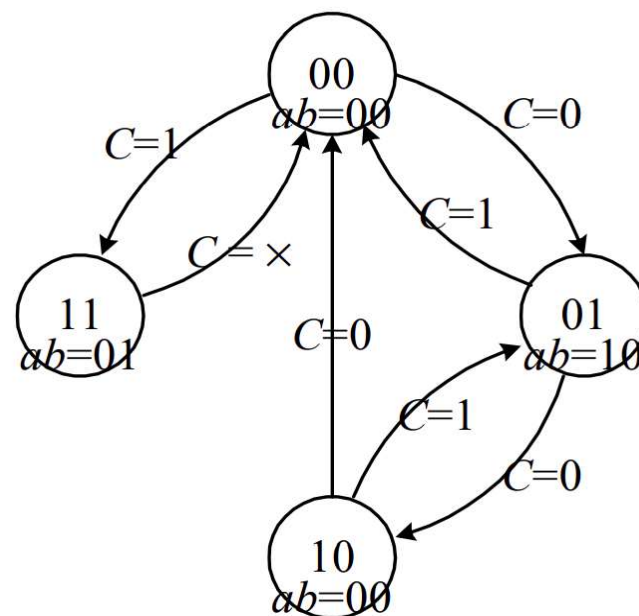
```
module fsm_VD2 (ck,rs,c,ab);  
parameter s0 = 2'b00;  
parameter s1 = 2'b01;  
parameter s2 = 2'b10;  
parameter s3 = 2'b11;  
input ck,rs,c;  
output [1:0]ab;  
reg [1:0]current_state, next_state;  
reg [1:0]ab;
```

```
always @ (*)  
begin  
case (current_state)  
s0 : if (c) next_state = s3;  
     else next_state = s1;  
s1 : if (c) next_state = s0;  
     else next_state = s2;  
s2 : if (c) next_state = s1;  
     else next_state = s0;  
s3 : next_state = s0;  
default next_state = s0;  
endcase  
end
```



```
always @ (posedge ck or posedge rs)
begin
  if (rs == 1) current_state <= s3;
  else current_state <= next_state;
end
```

```
always @ (*)
case (current_state)
  s0 : ab = 2'b00;
  s1 : ab = 2'b10;
  s2 : ab = 2'b00;
  s3 : ab = 2'b01;
endmodule
```



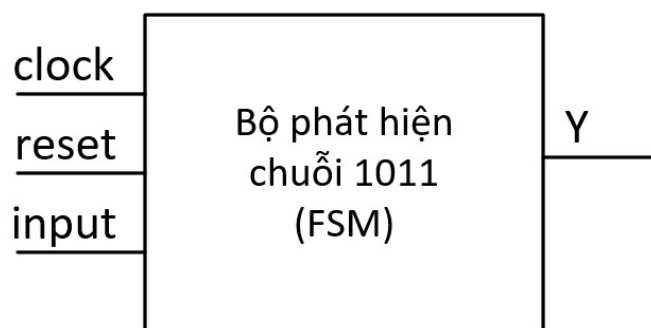
3. Thiết kế FSM có chức năng phát hiện chuỗi

Ví dụ 3: Mô tả máy trạng thái hữu hạn (FSM) có chức năng nhận biết chuỗi 1011 từ ngõ vào input. Chân Ck tác động theo cạnh lên, chân RS không đồng bộ và tích cực mức cao. Khi phát hiện đúng chuỗi 1011 thì ngõ ra Y sẽ bằng 1.

Giải:

B1: Xác định số ngõ vào, ngõ ra

Gán chân:



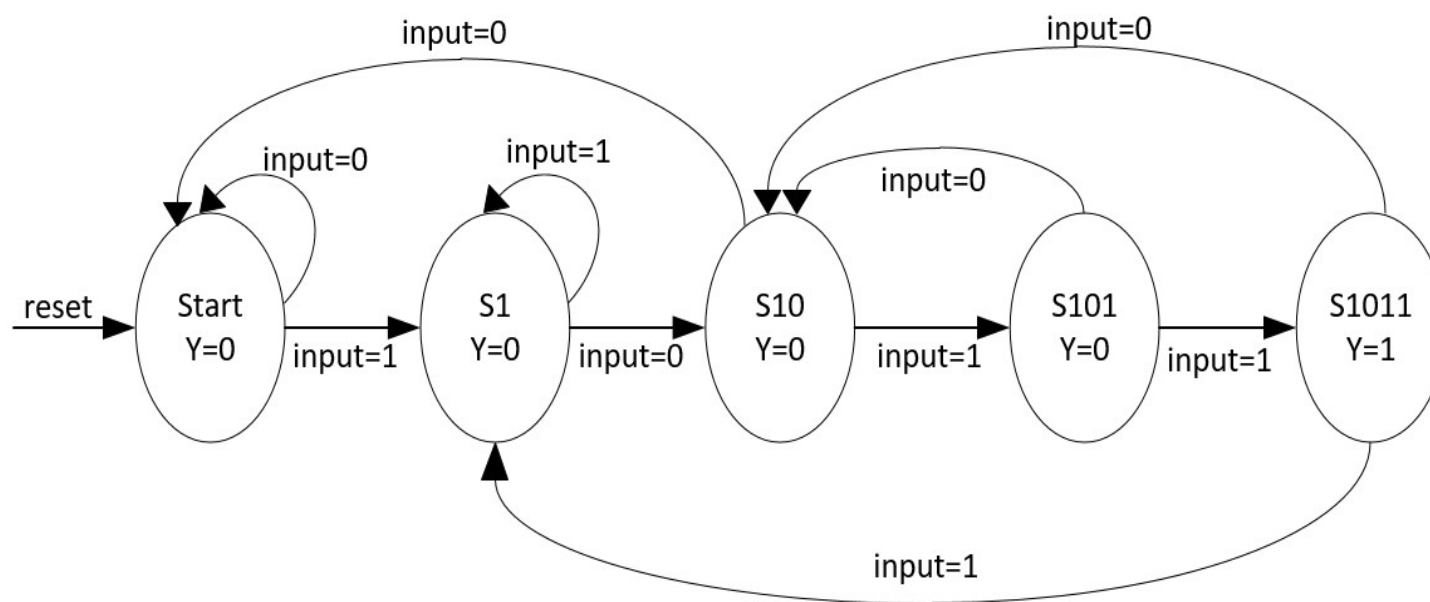
clock : SW[0]

reset : SW[1]

input : SW[2]

y : LEDR[0]

B2: Graph trạng thái dạng moore



B3: Viết chương trình dùng ngôn ngữ Verilog

```
module bai1 (SW,LEDR);  
  input [2:0]SW;  
  output reg [0:0]LEDR;  
  parameter start      = 3'b000;  
  parameter s1         = 3'b001;  
  parameter s10        = 3'b010;  
  parameter s101       = 3'b011;  
  parameter s1011      = 3'b100;  
  reg [2:0]current_state, next_state;
```

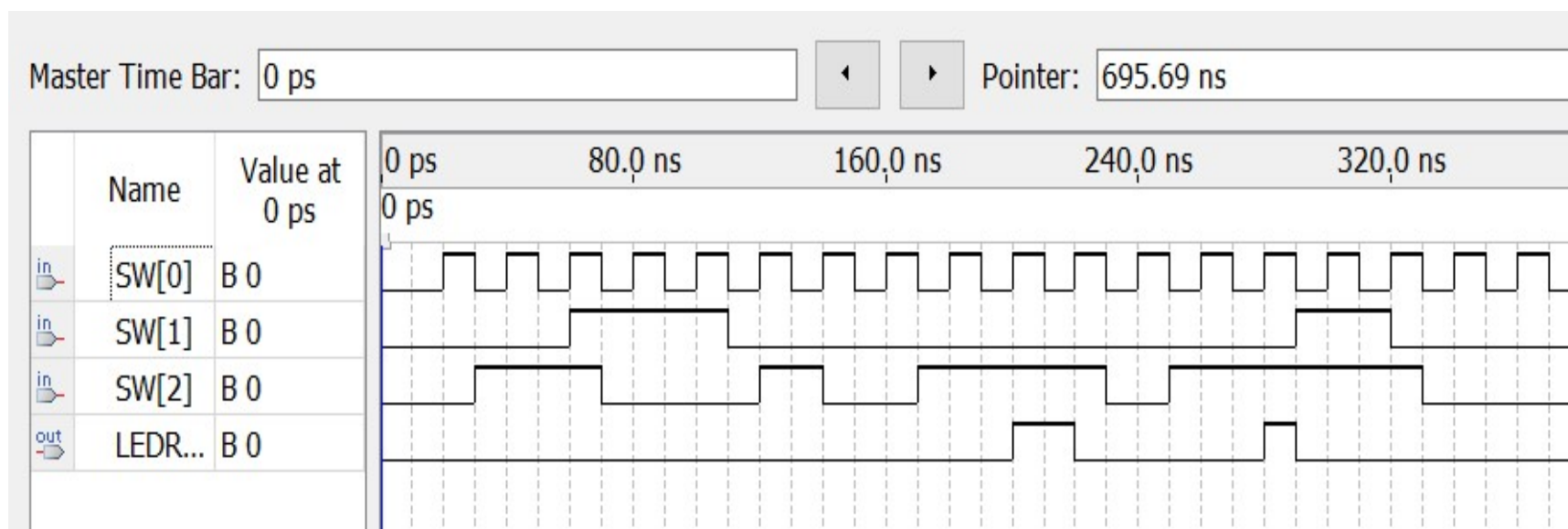
```
  always @ (*)  
  begin  
    case (current_state)  
      start : if (SW[2]) next_state = s1;  
              else      next_state =  
current_state;  
      s1    : if (~SW[2]) next_state = s10;  
              else      next_state =  
current_state;  
      s10   : if (SW[2]) next_state = s101;  
              else next_state = start;  
      s101  : if (SW[2]) next_state = s1011;
```

```
        else next_state = s10;  
s1011 : if (SW[2]) next_state = s1;  
        else next_state = s10;  
default next_state = start;  
endcase  
end
```

```
always @ (posedge SW[0] or posedge  
SW[1])  
begin  
    if (SW[1]) current_state <= start;  
    else current_state <= next_state;  
end  
//mach to hop tac dong trang thai ngo ra fsm  
always @ (*)  
begin  
    if (current_state == s1011) LEDR[0] = 1'b1;  
    else LEDR[0] = 1'b0;  
end  
endmodule
```

B4 : Biên dịch để phân tích, tổng hợp và tạo ra file .sof

B5 : Mô phỏng và kiểm tra kết quả



B6 : Gán chân cho FPGA

B7 : Nạp file.sof vào FPGA, kiểm tra hoạt động của mạch.

4. Lập trình ứng dụng dùng FSM

VD 4: Mô tả máy trạng thái hữu hạn (FSM) thực hiện mạch đếm xuống 5 về 0, ngõ ra hiển thị trên 1 led bảy đoạn loại anode chung. Chân Ck tác động theo cạnh lên, chân RS không đồng bộ và tích cực mức cao, Thời gian hiển thị một chữ số là 1s, khi RS tích cực thì các led hiển thị số 5

Giải:

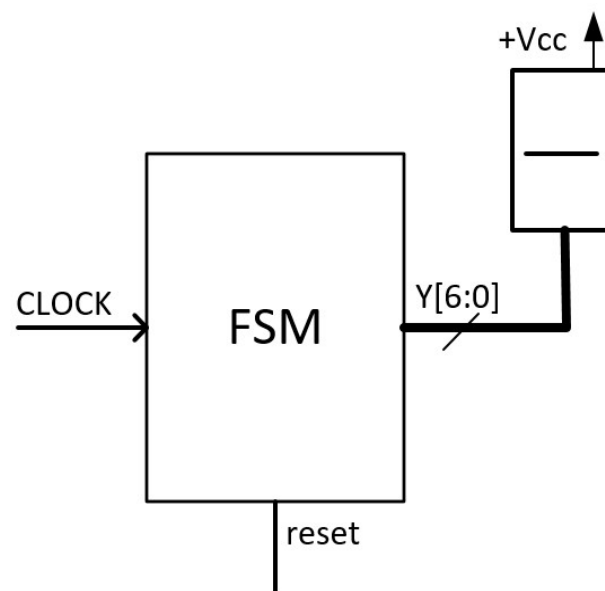
Sơ đồ chân in/out:

Gán chân:

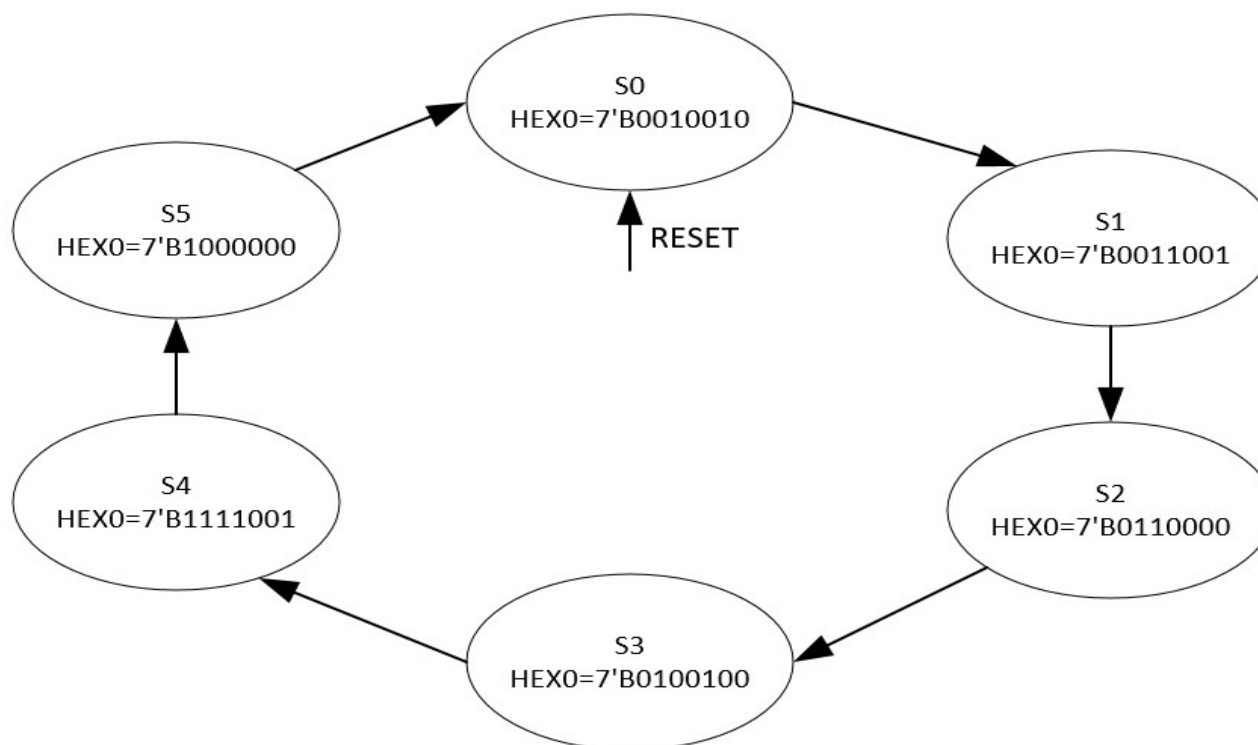
clock : CLOCK_50

reset : SW[0]

Y[6:0] : HEX0[6:0]



Graph trạng thái dạng moore



Chọn xung ck có $f = 1\text{Hz}$

```

module bai1 (SW,CLOCK_50,HEX0);
input [0:0]SW;
input CLOCK_50;
output reg [6:0]HEX0;
parameter s0      = 3'b000;
parameter s1      = 3'b001;
parameter s2      = 3'b010;
parameter s3      = 3'b011;
parameter s4      = 3'b100;
parameter s5      = 3'b101;
reg [2:0]current_state, next_state;
integer q;
reg clock_1s = 1'b0;
always @ (posedge CLOCK_50)
    begin
        q=q+1;
        if (q == 49999999)
            begin
                clock_1s = ~clock_1s;
                q = 0;
            end
    end
end

```

```

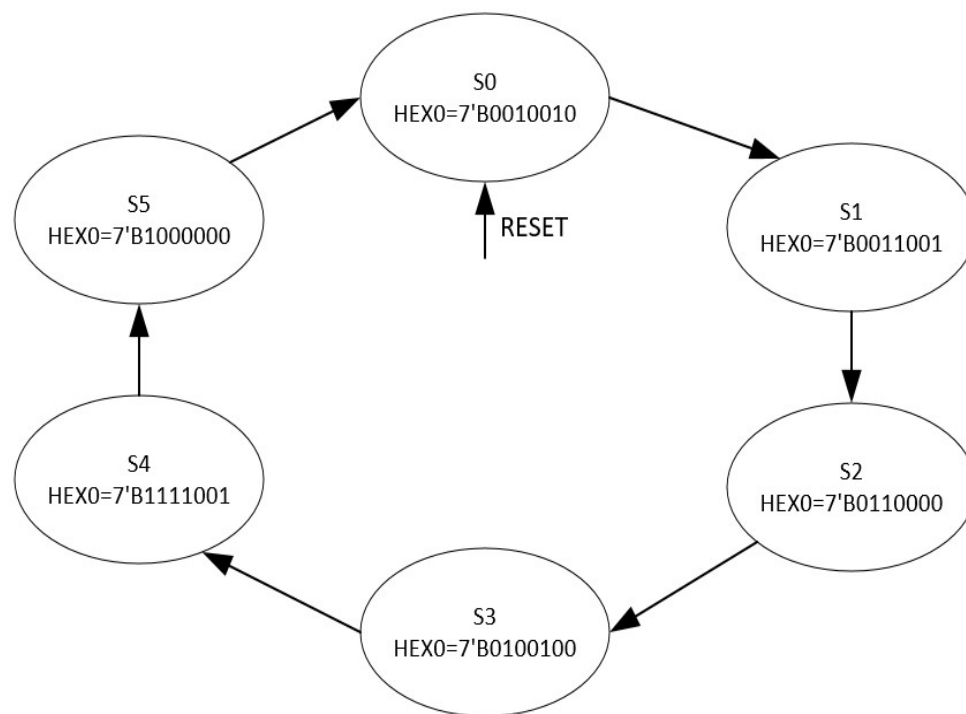
always @ (*)
begin
    case (current_state)
        s0 :    next_state = s1;
        s1 :    next_state = s2;
        s2 :    next_state = s3;
        s3 :    next_state = s4;
        s4 :    next_state = s5;
        s5 :    next_state = s0;
        default next_state = s0;
    endcase
end
//mach tuan tu tao trang thai trong cho fsm
always @ (posedge clock_1s or posedge SW[0])
begin
    if (SW[0]) current_state <= s0;
    else current_state <= next_state;
End

```

```
//mach to hop tac dong trang thai ngo ra fsm  
always @ (*)  
begin
```

```
    case (current_state)  
    s0: HEX0 = 7'b0010010;  
    s1: HEX0 = 7'b0011001;  
    s2: HEX0 = 7'b0110000;  
    s3: HEX0 = 7'b0100100;  
    s4: HEX0 = 7'b1111001;  
    s5: HEX0 = 7'b1000000;  
    endcase
```

```
end  
endmodule
```



Cảm ơn các bạn đã lắng nghe!

